

PostgreSQL

Binary Log

Cuando la base de datos va a realizar una escritura, el software de la base de datos va a ver en memoria. Por rendimiento también tenemos datos e índices en memoria principal. La memoria principal es volátil. Es decir que si hay un error (se va la corriente, se daña), lo que pasa es que se pierden los datos. La memoria de datos y de índices se organiza en file System.

El file system tiene archivos random access. Esto quiere decir que a veces la cabeza lectora escribe en diferentes partes. Las operaciones de escritura a disco son lentas. Si hay muchas escrituras, para cumplir con ACID también hay que actualizar memoria inmediatamente y el file system. Las operaciones hacia disco causa que todo se vuelva lento. Y si no tenemos binary log y queremos cumplir con ACID, todo va a ser mucho más lento.

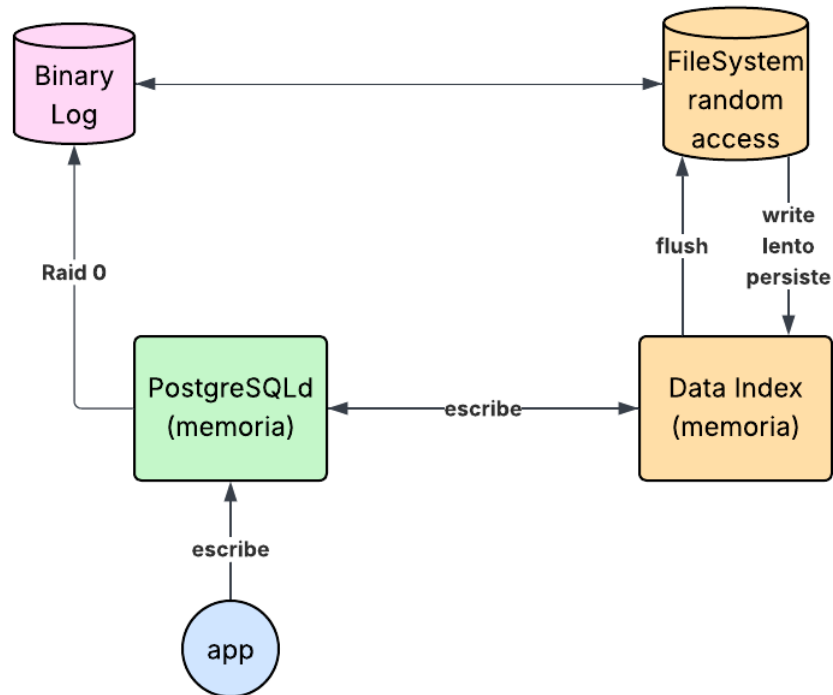


Figure 1: Diagrama1

Una buena práctica es tener lo más separados posibles los discos. Se puede tener un disco separado de binary log, para que esté separado de los datos de la base de datos, aunque suelen estar dentro del mismo disco.

El binary log es un archivo de acceso secuencial, y en el momento que entra una transacción, se asigna un id de transacción. La transacción es una operación de modificación de datos, no una secuencia de statements que se comportan como uno solo. Siempre viene un commit al finalizar la transacción.

Siempre se hace “append” en el binary log, se pega al final. Así sabemos que es secuencial, y el primero se insertó antes que el último. La diferencia del almacenamiento de binary log y el almacenamiento file system, es que en el binary log es un solo archivo y se hace append. En cambio, en el file system, hay tablas con archivos separados, y la inserción sigue las reglas de una base de datos relacional

Luego se hace la actualización de los datos en memoria principal. Esto no es lo mismo que escribir en la base de datos. Los datos quedan en memoria principal, es decir, en un buffer. El proceso de base de datos sabe que los datos están en memoria principal, pero los mantiene ahí por un tiempo.

Cada cierto tiempo la base de datos hace un flush de los datos. Es decir, agarra los datos de memoria principal y los persiste en disco. Inmediatamente que hace esto, pone commit en las transacciones. Esto garantiza que los datos son persistentes.

Si la base de datos se cae antes de hacer este flush, se va a fijar a cuáles no se les hizo commit. Esto significa que la transacción no llegó a disco. Así que va a busca en el binary log las transacciones que faltan de hacer commit, y secuencialmente les va haciendo commit una por una.

El binary log asegura velocidad y persistencia.

Muchas bases de datos utilizan el binary log como un sistema de replicación de datos. Siempre hay una base de datos activa y otra standby. La standby va reflejando la activa. Si alguien hace una escritura, se pasa la entrada al binary log. Si ambas bases guardan los datos en el binary log, siempre va a haber una manera de replicar los datos.

Es importante ser pesimista y pensar en el peor escenario.

Si se tiene un esquema con un servidor en standby esperando que el primario falle, se necesita el mismo poder en ambos servidores. Siempre que en un servidor se corre una base de datos, no se corre ningún otro proceso, porque cualquier aplicación compite por los recursos de la computadora.

A veces la solución para arreglar la base de datos es el switch over, pero si se tiene una aplicación, hay que limpiar el disco. La base de datos puede quedar corrupta, y hay que buscar soluciones distintas para levantar la base de datos. Por ejemplo, volver a interpretar el binary log corrupto hasta que se recuperen

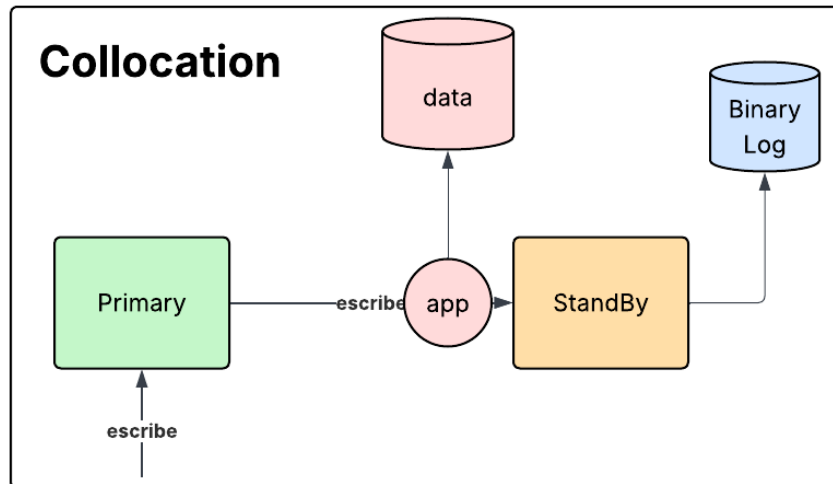


Figure 2: Diagrama2

los datos. Esta es la gran importancia del binary log, puede recuperar una base de datos en caso de desastre.

Master-Slave Replication

Se tiene un modo asíncrono y un modo síncrono.

El modo asíncrono quiere decir que se va a hacer un envío en batch. Esto quiere decir que se completa una ventana de eventos en el binary log. Cuando se recoge cierta cantidad de eventos se hace el envío. El slave va a estar atrás del binary log del master por un tiempo, pero depende de lo que el negocio aguante.

El modo síncrono quiere decir que inmediatamente llega un evento al binary log, se envía al slave.

Al generar información batch se necesita el uso de red solo una vez para mucha información. Para la información síncrona se usa la red cada vez que hay un evento. Hay un mayor ahorro de recursos en el modo asíncrono.

El riesgo de estar en modo asíncrono, es que si algo sale mal, hay mucha información que no va a haber llegado.

Se hace un switch over que permite cambiar el tráfico si la base de datos falla, y si no, hay que implementar un fail over. El fail over necesita un testigo. El testigo desempata el voto cuando se hace el fail over automático. Cuando no se pueden tener 3 servidores, se tienen 2 y un testigo, que es una instancia muy

pequeña. Su trabajo es correr healthchecks, que determina si un servidor se falló.

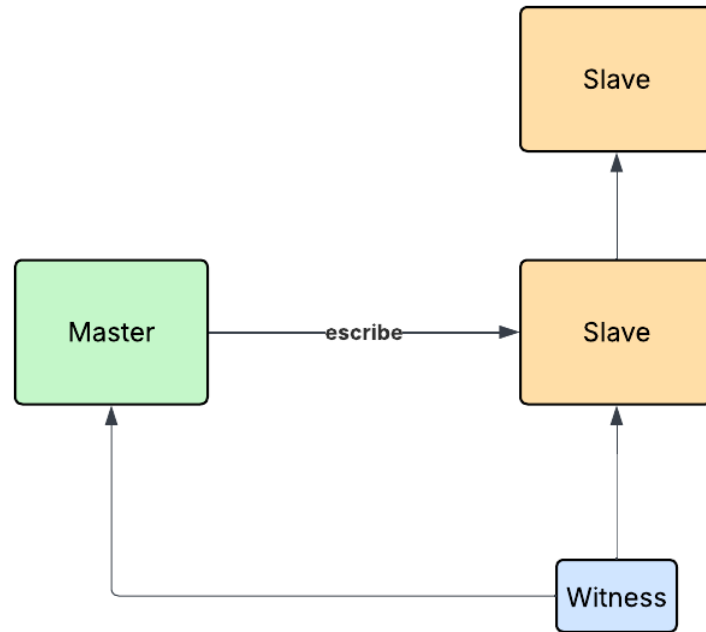


Figure 3: Diagrama3

Las bases de datos modernas usan partitions o charts. Esto es una forma de dividir una tabla en pedazos. Se suele hacer basado en una columna, como el año del id. Asegura máximo uso de recursos simultáneos. Pero genera overhead.

El overhead quiere decir que se debe hacer una tarea, pero se gasta un tiempo extra en coordinación para hacer la tarea. La CPU, red y discos se usan para almacenar datos, pero se necesita comunicación a través de la red y particiones. Esto hace que se consuman recursos en forma de overhead.

Es importante recordar que la industria en vez de usar slave, ahora usa un read only replica. Nunca recibe actualizaciones, pero sí consultas. La actualización va a la base de datos encargada de write.

En este caso se tiene consistencia eventual. Tarda cierto tiempo en enviarse. El modelo no funciona para una base de datos SQL, porque una inconsistencia por una fracción de tiempo es un gran problema.

Si se quiere una consistencia fuerte se tiene un disco de data y uno de binary log. Cuando se pide una escritura, el cliente espera una respuesta. Cuando hay

un binary log, entonces se escribe primero en el binary log. En el momento de escribir en el binary log los datos son persistentes, y desde ese momento se le puede dar un mensaje de éxito al cliente. Pero hay que esperar a asegurarse que se sepa que el update que pide el cliente sí cumple con las reglas de la base de datos, por esto hay que esperar a escribir a memoria principal. No hay que esperar a que se haga flush.

Se necesita un strong consistency, por lo que hay que hacer los pasos para asegurarse que todo esté bien. Entre más slaves, más tiempo, y más overhead.

Warm StandBy using Point-In-Time Recovery

Se tienen dos slaves, y se manda información de un servidor a otro. Si se hace un data analytics, se puede ofuzcar la base de datos y se pueden tener fallos. En este caso, se rompe el vínculo entre las bases de datos, y así se evitan problemas si se cae la base de datos.

Cuando se reestablece el vínculo, se envía información de golpe del binary log a la base de datos, hasta que se recupere al punto en que esté igual a la principal.

Logical Replication (Only changes)

Replicación lógica implica hacer replicación con mecanismo publisher-susbscriber. La base de datos principal se convierte en un publisher, y cada base de datos secundaria es un subscriber. Una se puede suscribir a la tabla de estudiantes pero no a todas las tablas, y puede pedir actualización cada 3 meses. Otra base de datos puede suscribirse a la tabla de cursos y se actualiza cada 5 meses.

Trigger Base

Se tiene una base de datos principal y varias secundarias, que se suscriben a una tabla. Se espera a un trigger para meter los datos a data warehouse.

SQL Replication con Middleware

Los servidores no saben que los otros existen. En medio tienen un interceptor que es una pieza de software que se para enfrente de los servidores. Implementa la misma interfaz que implementa la base de datos. Las insert y update se envían a todos los servidores, y los servidores hacen lo mismo del binary log. Los select entran a load balancer y se envía a cualquiera. Se tiene garantía que la consistencia es fuerte.

Asynchrhonous Multimaster Replicacion y Synchronous Master Replication

Uno o más servidores son Master, y tienen la misma información. Ambas master esriben simultáneamente.

En los asíncronos, a veces se sincronizan cada hora, y al sincronizar hace merge y resuelve conflictos.